

Code-Layout, Readability and Reusability

Date	02 July 2026
Team ID	
Project Name	Rising Waters – A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability

This document evaluates the quality of the Rising Waters codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures the project is maintainable and can be extended in the future.

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed
1	Consistent Indentation	Uniform 4-space indentation used throughout app.py and the notebook	Yes
2	Meaningful Naming	Clear names used (FIELD_META, FIELD_GROUPS, MODEL, SCALER, FEATURE_NAMES)	Yes
3	Modular Structure	Data, notebook, model, and app are separated into dedicated folders	Yes
4	Comments & Docstrings	app.py has a module docstring describing routes and run instructions	Yes
5	Configuration Separation	Field metadata (labels/units/help text) kept in a single FIELD_META dict, not hard-coded in templates	Yes
6	Reusable Model Bundle	Model, scaler and feature names are bundled together via joblib so the app needs no retraining logic	Yes
7	Template Inheritance	All pages extend a shared base.html to avoid duplicated markup	Yes
8	Hard-coded Values / Paths	One route (hero_image) references a local absolute Windows path, which should be replaced with a static asset before production deployment	Partial
9	Error Handling	Form parsing wraps float conversion in try/except and re-renders the form with field-level errors	Yes

Remarks

The codebase is compact and clearly organized (data/, notebook/, model/, app/). The single notable cleanup item is the hard-coded local file path in the /hero_image route in app.py, which should be moved into app/static/ before production deployment for portability.

